

datalex



# Medallion Architecture Use Case



# Alexandre BERGERE

Head of Data & AI Engineer & Partners at DataGalaxy, Data Architect independent
Delta & openLineage lover

**Data Expert**
(Audit, Modeling, Urbanization, BI's project – Azure, Big Data, Spark, openLineage & Delta lover)

**Trainer**
(BI, Big Data, MongoDB, NoSQL, SQL, Cloud Computing)

**Speaker**
(Pass Summit 2018, Spark AI Tour 2020, …)

**Cloud & data tech articles**
(medium, LinkedIn, slideshare)

## Avanade
Sr Anls, Data Analyst

2016 - 2019

## Trainer
2016 - x

ESAIP, ESME, ECS, NEOMA, ESEO, EFREI, Sorbonne Paris

*BI, Big Data, MongoDB, NoSQL, Panorama de la data, Cloud computing*

## OpenClassroom
Mentor

2019 - x

## Datalex
2019 - x

Data Architect Freelance

## DataRedKite
CTO & Co-Founder

2020 - 2023

## DataGalaxy
2021 - x

Head of Data & AI Engineer

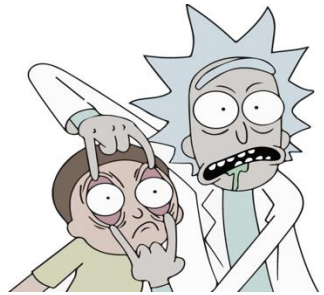
# Don’t get lost




Practice



Demo



Important

# Learning Objectives

1. Understand OLAP / Fact / Dimension Tables

2. Analyze data in a relational data warehouse

# Subsystems of ETL

revisited by Kimball Group



# Subsystems of ETL

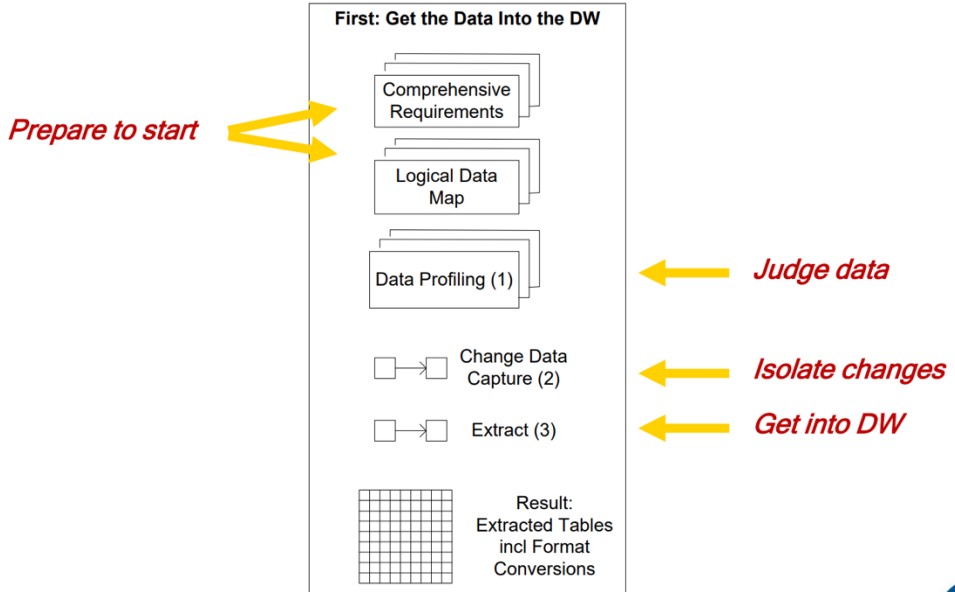
Best practices into 34 subsystems that represent the key ETL architecture components required in almost every dimensional data warehouse environment.

- **EXTRACTING: GETTING DATA INTO THE DATA WAREHOUSE**
- **CLEANING AND CONFORMING DATA**
- **DELIVERING: PREPARE FOR PRESENTATION**
- **MANAGING THE ETL ENVIRONMENT**

https://www.kimballgroup.com/2007/10/subsystems-of-etl-revisited/

# Extracting data from source systems

- Data Profiling (subsystem 1) – Explores a data source to determine its fit for inclusion as a source and the associated cleaning and conforming requirements.
- Change Data Capture (subsystem 2) – Isolates the changes that occurred in the source system to reduce the ETL processing burden.
- Extract System (subsystem 3) – Extracts and moves source data into the data warehouse environment for further processing.

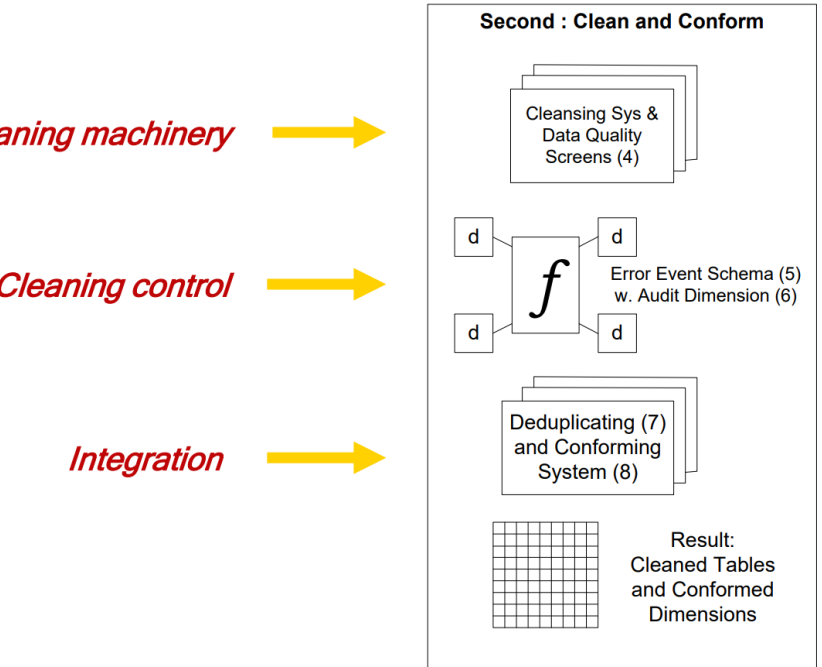




https://www.kimballgroup.com/2007/10/subsystems-of

# Cleaning and Conforming Data

- Data Cleansing System (subsystem 4) – Implements data quality processes to catch quality violations.
- Error Event Tracking (subsystem 5) – Captures all error events that are vital inputs to data quality improvement.
- Audit Dimension Creation (subsystem 6) – Attaches metadata to each fact table as a dimension. This metadata is available to BI applications for visibility into data quality.
- Deduplication (subsystem 7) – Eliminates redundant members of core dimensions, such as customers or products. May integration across multiple sources and application of survi rules to identify the most appropriate version of a duplicate
- Data Conformance (subsystem 8 ) – Enforces common dim attributes across conformed master dimensions and cc metrics across related fact tables.





https://www.kimballgroup.com/2007/10/subsystems-of-etl-r

# Delivering : prepare for presentation

- Slowly Changing Dimension (SCD) Manager (subsystem 9) – Implements logic for slowly changing dimension attributes.
- Surrogate Key Generator (subsystem 10) – Produces surrogate keys independently for every dimension.
- Hierarchy Manager (subsystem 11) – Delivers multiple, simultaneous, embedded hierarchical structures in a dimension.
- Special Dimensions Manager (subsystem 12) – Creates placeholders in the ETL architecture for repeatable processes supporting an organization's specific dimensional design characteristics, including standard dimensional design constructs such as junk dimensions, mini-dimensions and behavior tags.
- Fact Table Builders (subsystem 13) – Construct the three primary types of fact tables: transaction grain, periodic snapshot and accumulating snapshot.
- Surrogate Key Pipeline (subsystem 14) – Replaces operational natural keys in the incoming fact table record with the appropriate dimension surrogate keys.
- Multi-Valued Bridge Table Builder (subsystem 15) – Builds and maintains bridge tables to support multi-valued relationships.



https://www.kimballgroup.com/2007/10/subsystems-of-etl-revisited/

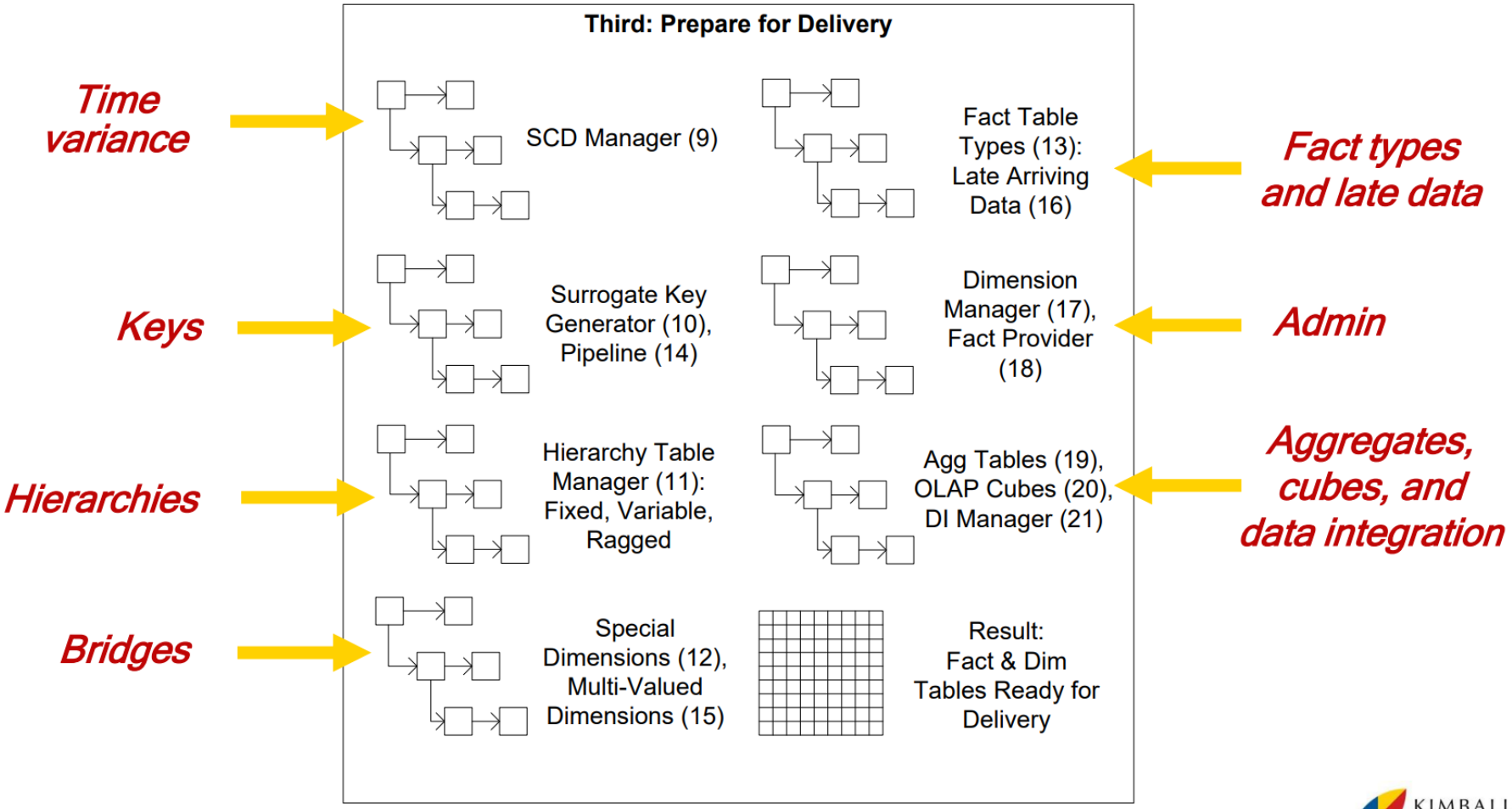
# Delivering : prepare for presentation

- Late Arriving Data Handler (subsystem 16) – Applies special modifications to the standard processing procedures to deal with late-arriving fact and dimension data.
- Dimension Manager (subsystem 17) – Centralized authority who prepares and publishes conformed dimensions to the data warehouse community.
- Fact Table Provider (subsystem 18) – Owns the administration of one or more fact tables and is responsible for their creation, maintenance and use.
- Aggregate Builder (subsystem 19) – Builds and maintains aggregates to be used seamlessly with aggregate navigation technologies for enhanced query performance.
- OLAP Cube Builder (subsystem 20) – Feeds data from the relational dimensional schema to populate OLAP cubes.
- Data Propagation Manager (subsystem 21) – Prepares conformed, integrated data from the data warehouse presentation server for delivery to other environments for special purposes.



https://www.kimballgroup.com/2007/10/subsystems-of-etl-revisited/

# Delivering : prepare for presentation




© 2013 Kimball Group. All rights reserved.

KIMBALL GROUP



https://www.kimballgroup.com/2007/10/subsystems-of-etl-revisited/

# Managing the ETL environment

A data warehouse will not be a success until it can be relied upon as a dependable source for business decision making. To achieve this goal, the ETL system must constantly work toward fulfilling three criteria:

- *Reliability*. The ETL processes must run consistently to provide data on a timely basis that is trustworthy at any level of detail.
- *Availability*. The data warehouse must meet its service level agreements. The warehouse should be up and available as promised.
- *Manageability*. A successful data warehouse is never done; it constantly grows and changes along with the business. Thus, ETL processes need to evolve gracefully as well.



https://www.kimballgroup.com/2007/10/subsystems-of-etl-revisited/

# Managing the ETL environment

- Job Scheduler (subsystem 22) – Reliably manages the ETL execution strategy, including the relationships and dependencies between ETL jobs.
- Backup System (subsystem 23) – Backs up the ETL environment for recovery, restart and archival purposes.
- Recovery and Restart (subsystem 24) – Processes for recovering the ETL environment or restarting a process in the event of failure.
- Version Control (subsystem 25) – Takes snapshots for archiving and recovering all the logic and metadata of the ETL pipeline.
- Version Migration (subsystem 26) – Migrates a complete version of the ETL pipeline from development into test and finally into production.
- Workflow Monitor (subsystem 27) – Ensures that the ETL processes are operating efficiently and that the warehouse is being loaded on a consistently timely basis.
- Sorting (subsystem 28) – Serves the fundamental, high-performance ETL processing role.



https://www.kimballgroup.com/2007/10/subsystems-of-etl-revisited/

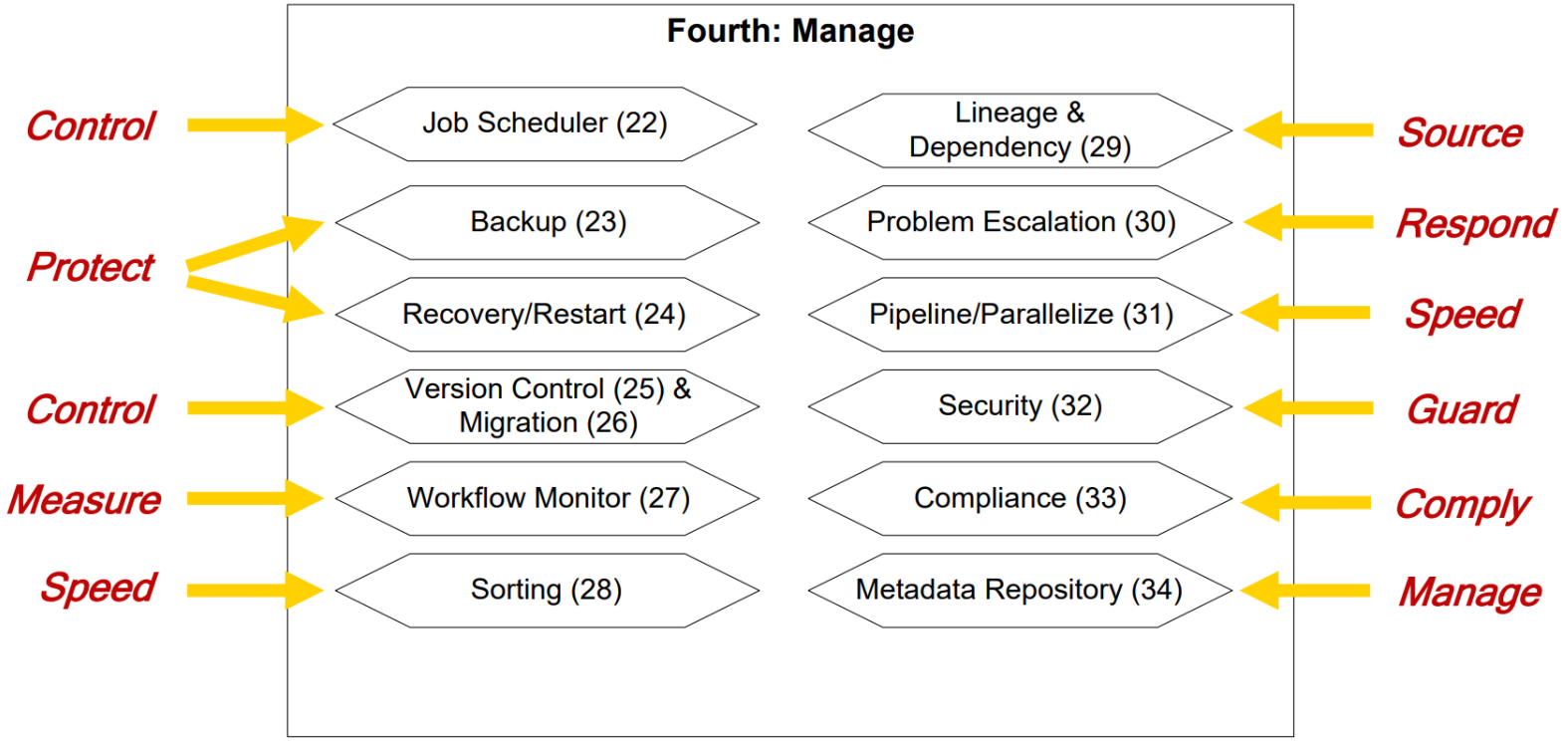
# Managing the ETL environment

- Lineage and Dependency (subsystem 29) – Identifies the source of a data element and all intermediate locations and transformations for that data element or, conversely, start with a specific data element in a source table and reveals all activities performed on that data element.
- Problem Escalation (subsystem 30) – Support structure that elevates ETL problems to appropriate levels for resolution.
- Paralleling and Pipelining (subsystem 31) – Enables the ETL system to automatically leverage multiple processors or grid computing resources to deliver within time constraints.
- Security (subsystem 32) – Ensures authorized access to (and a historical record of access to) all ETL data and metadata by individual and role.
- Compliance Manager (subsystem 33) – Supports the organization's compliance requirements typically through maintaining the data's chain of custody and tracking who had authorized access to the data.
- Metadata Repository (subsystem 34) – Captures ETL metadata including the process metadata, technical metadata and business metadata, which make up much of the metadata of the total DW/BI environment.



https://www.kimballgroup.com/2007/10/subsystems-of-etl-revisited/

# Managing the ETL environment



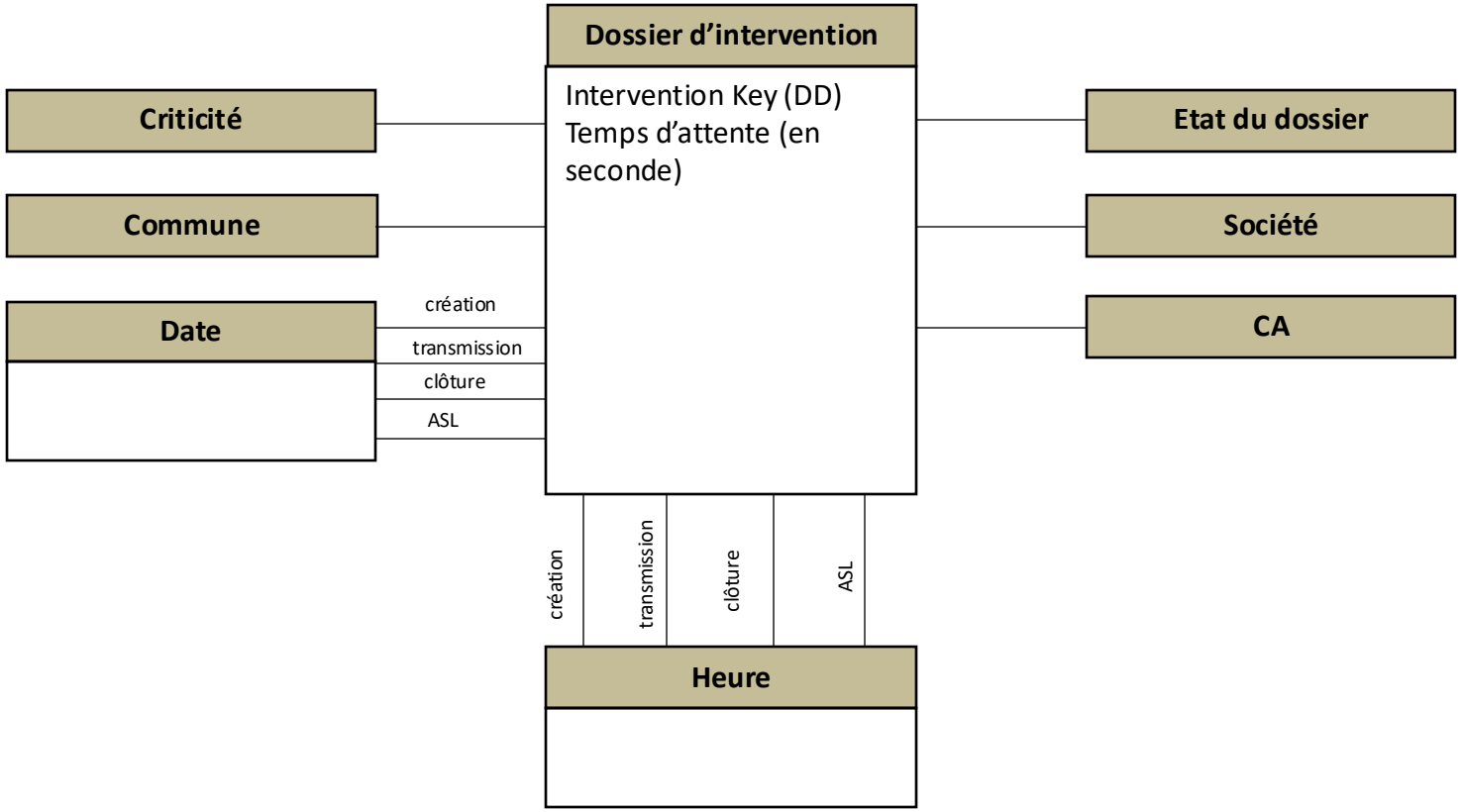

© 2013 Kimball Group. All rights reserved.

KIMBALL GROUP

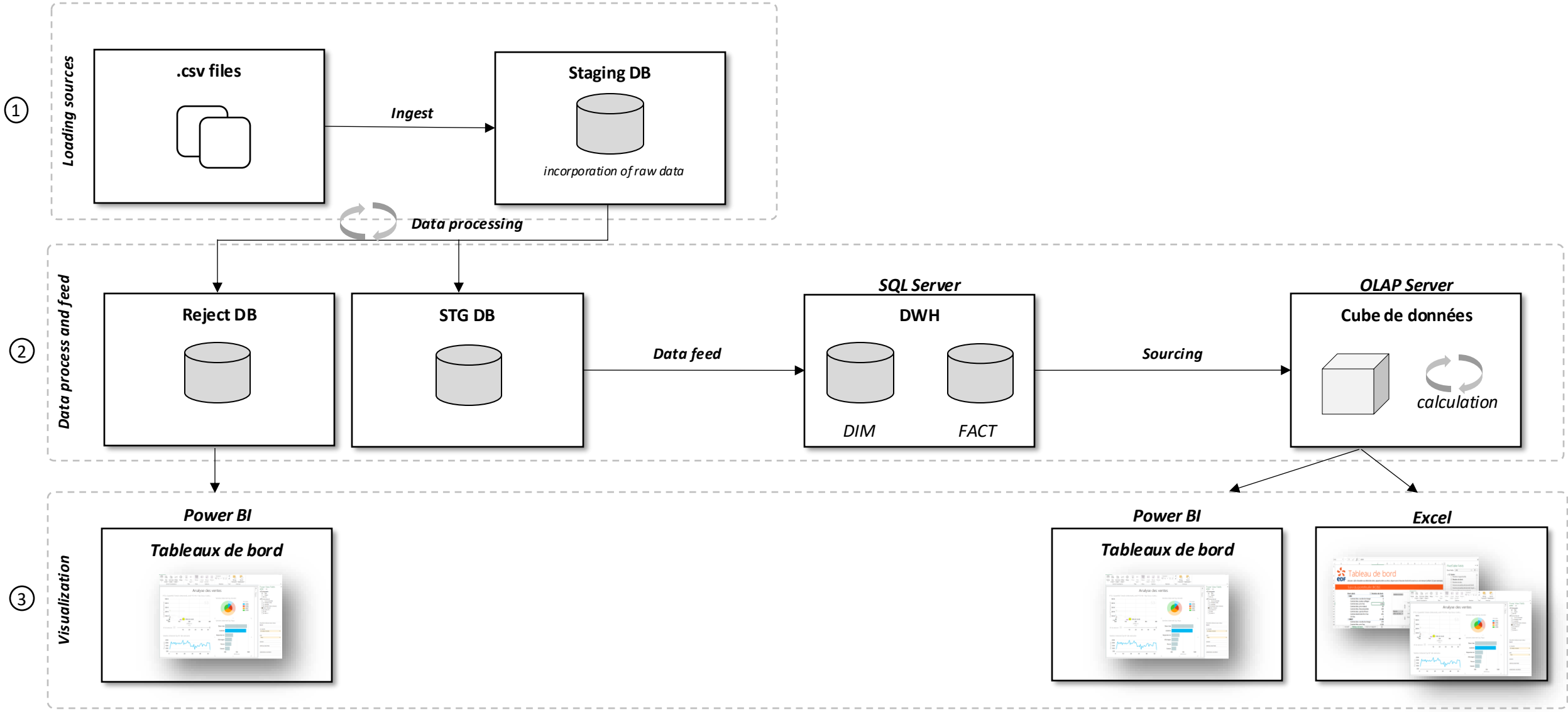
https://www.kimballgroup.com/2007/10/subsystems-of-etl-revisited/

# Suivi d’intervention – Use Case

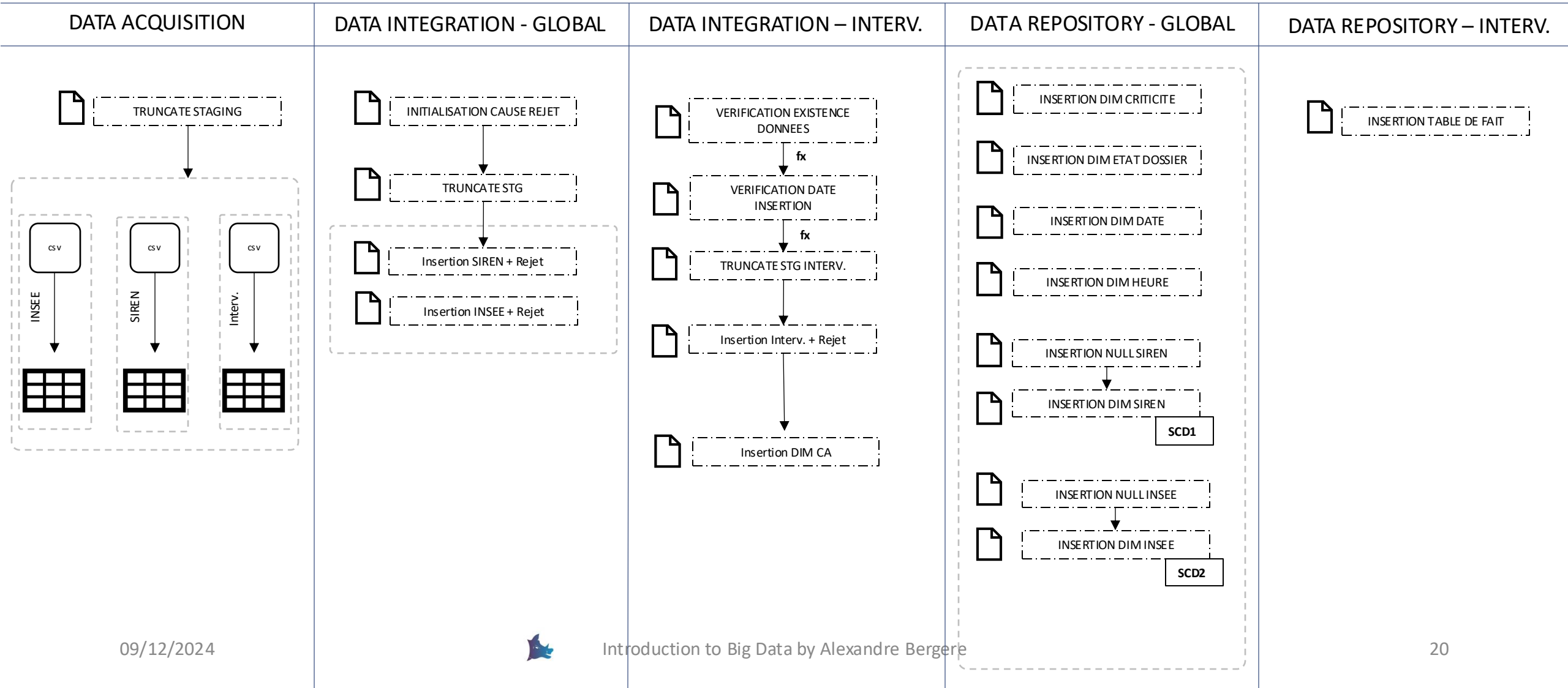
# Target modelisation



# BI Architecture



# BI Flow


DATA ACQUISITION
DATA INTEGRATION - GLOBAL
DATA INTEGRATION – INTERV.
DATA REPOSITORY - GLOBAL
DATA REPOSITORY – INTERV.
TRUNCATE STAGING
csv
INSEE
csv
SIREN
csv
Interv.
INITIALISATION CAUSE REJET
TRUNCATE STG
Insertion SIREN + Rejet
Insertion INSEE + Rejet
VERIFICATION EXISTENCE DONNEES
fx
VERIFICATION DATE INSERTION
fx
TRUNCATE STG INTERV.
Insertion Interv. + Rejet
Insertion DIM CA
INSERTION DIM CRITICITE
INSERTION DIM ETAT DOSSIER
INSERTION DIM DATE
INSERTION DIM HEURE
INSERTION NULL SIREN
INSERTION DIM SIREN
SCD1
INSERTION NULL INSEE
INSERTION DIM INSEE
SCD2
INSERTION TABLE DE FAIT

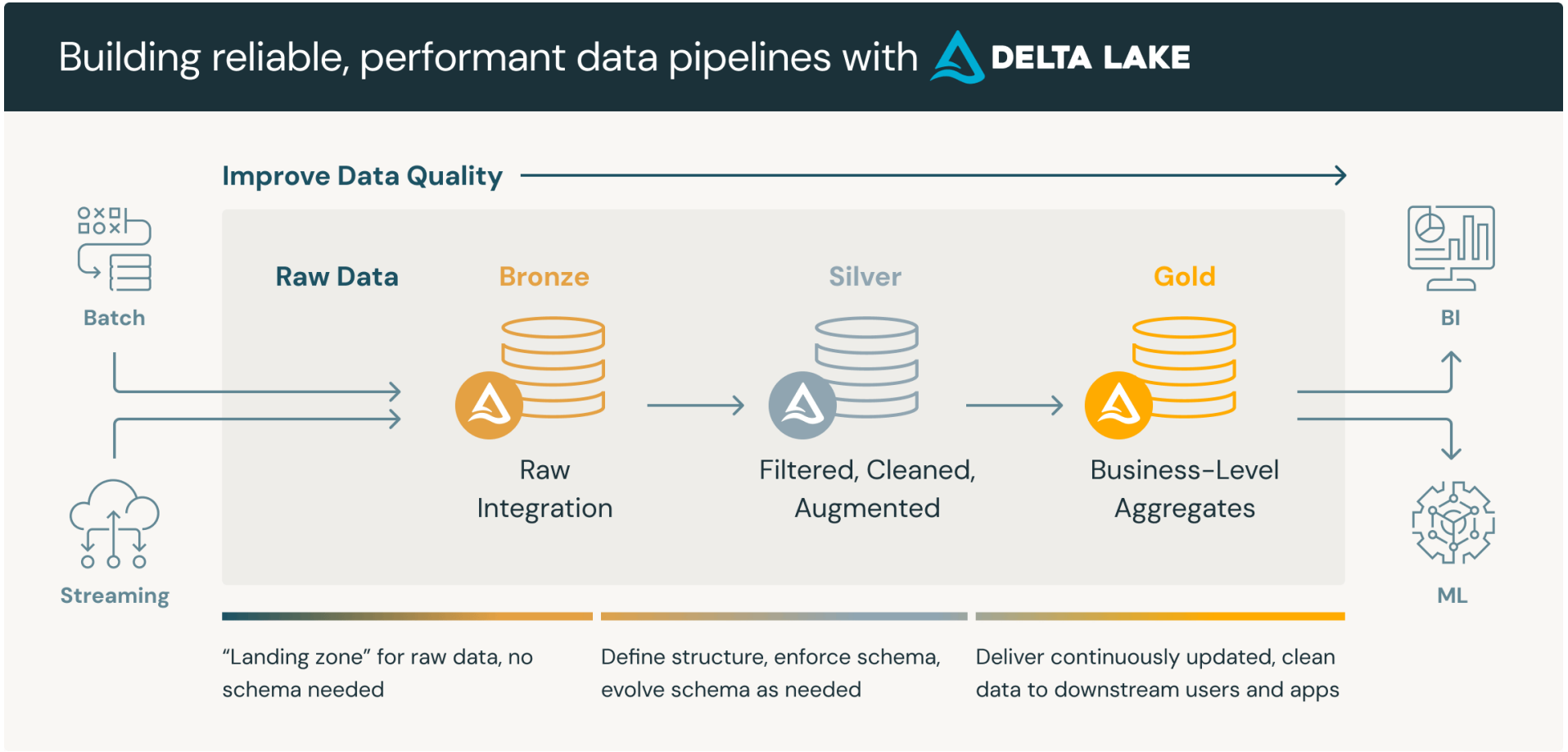
# Medallion Architecture

# What is a medallion architecture?

A **medallion architecture** is a data design pattern used to logically organize data in a lakehouse, with the goal of incrementally and progressively improving the structure and quality of data as it flows through each layer of the architecture (from Bronze ⇒ Silver ⇒ Gold layer tables). Medallion architectures are sometimes also referred to as "multi-hop" architectures.

https://www.databricks.com/glossary/medallion-architecture

# What is a medallion architecture?




https://www.databricks.com/glossary/medallion-architecture

# Bronze layer

## raw data

The **Bronze layer** is where we land all the data from external source systems.

The table structures in this layer correspond to the source system table structures "as-is," along with any additional metadata columns that capture the load date/time, process ID, etc.

The focus in this layer is quick Change Data Capture and the ability to provide an historical archive of source (cold storage), data lineage, auditability, reprocessing if needed without rereading the data from the source system.

https://www.databricks.com/glossary/medallion-architecture

# Silver layer

## cleansed and conformed data

In the **Silver layer** of the lakehouse, the data from the Bronze layer is matched, merged, conformed and cleansed ("just-enough") so that the Silver layer can provide an "Enterprise view" of all its key business entities, concepts and transactions. (e.g. master customers, stores, non-duplicated transactions and cross-reference tables).

The Silver layer brings the data from different sources into an Enterprise view and enables self-service analytics for ad-hoc reporting, advanced analytics and ML. It serves as a source for Departmental Analysts, Data Engineers and Data Scientists to further create projects and analysis to answer business problems via enterprise and departmental data projects in the Gold Layer

From a data modeling perspective, the Silver Layer has more 3rd-Normal Form like data models. Data Vault-like, write-performant data models can be used in this layer.

https://www.databricks.com/glossary/medallion-architecture

# Gold layer

curated business-level tables

Data in the **Gold layer** of the lakehouse is typically organized in consumption-ready "project-specific" databases.

The Gold layer is for reporting and uses more de-normalized and read-optimized data models with fewer joins. The final layer of data transformations and data quality rules are applied here.

Final presentation layer of projects such as Customer Analytics, Product Quality Analytics, Inventory Analytics, Customer Segmentation, Product Recommendations, Marking/Sales Analytics etc. fit in this layer.

A lot of Kimball style star schema-based data models or Inmon style Data marts fit in this Gold Layer of the lakehouse.

https://www.databricks.com/glossary/medallion-architecture

# Benefits of a lakehouse architecture

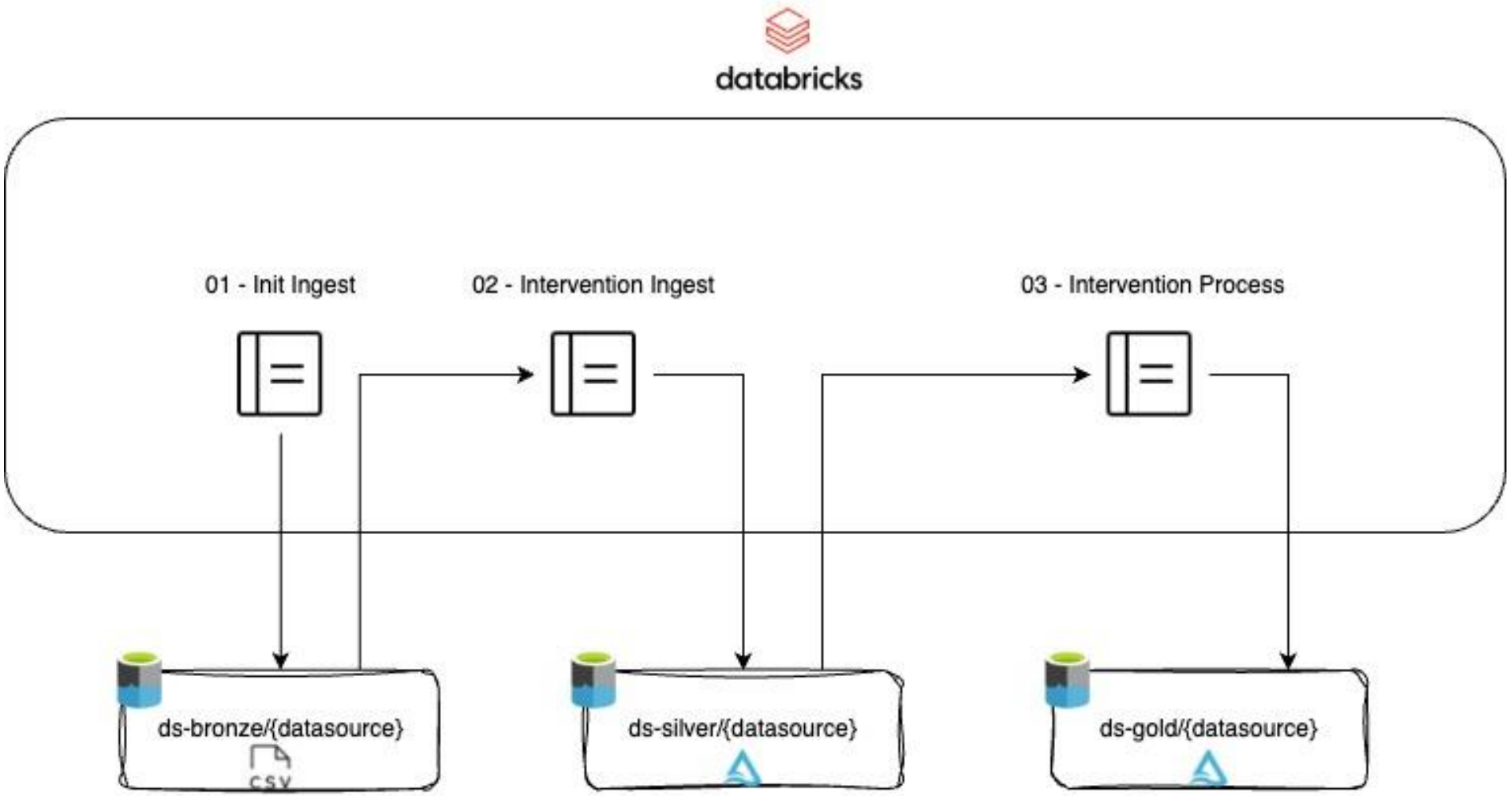
- Simple data model
- Easy to understand and implement
- Enables incremental ETL
- Can recreate your tables from raw data at any time
- ACID transactions, time travel

https://www.databricks.com/glossary/medallion-architecture

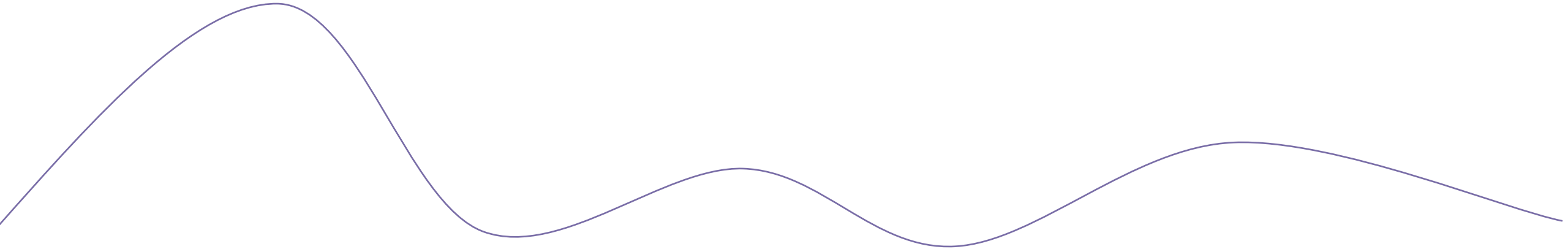
# Use Case – Data Flow Medallion Architecture



# Use Case – Applicative Flow Medallion Architecture



# Implementing Sources & Infrastructure

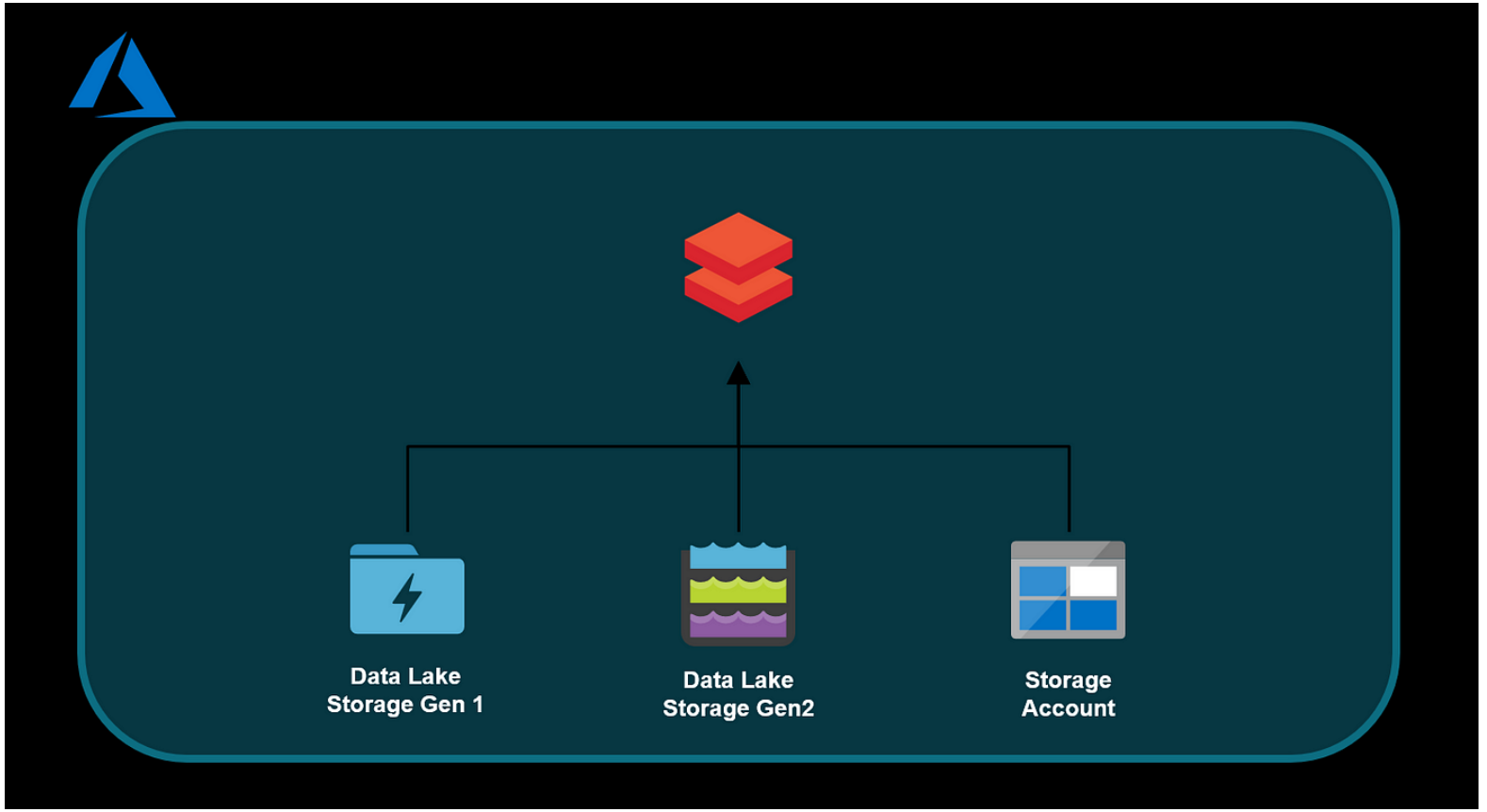


# Demo – Datalake Creation & Access



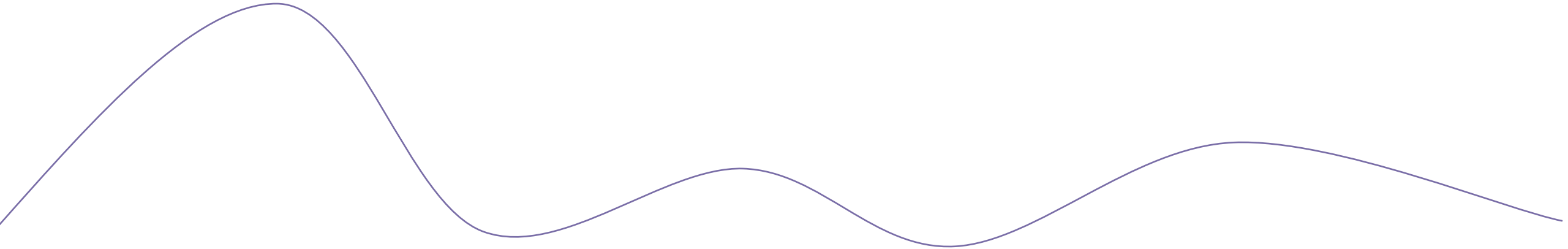
# Azure Databricks — Connect Azure storage to Databricks



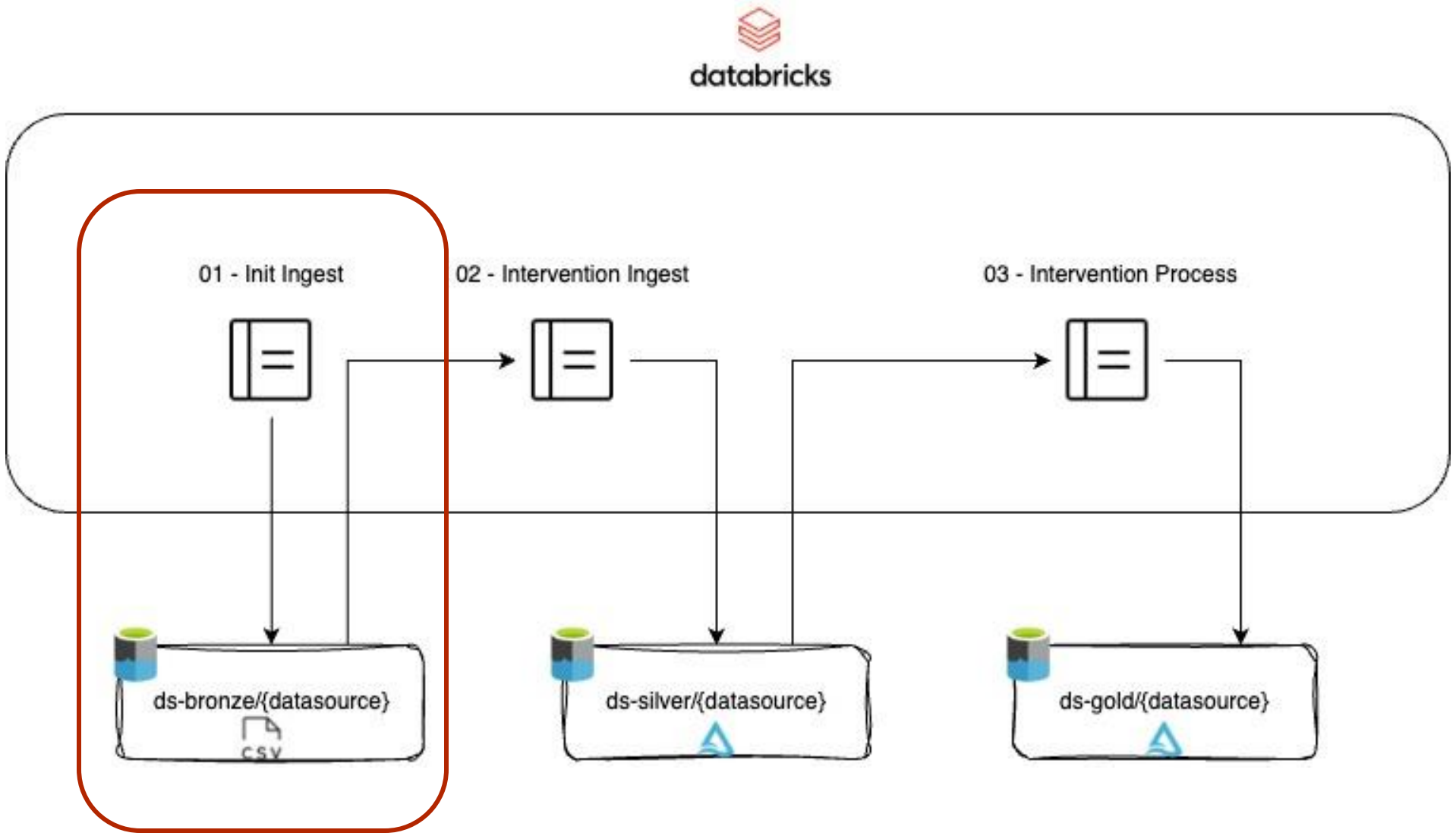



https://medium.com/datalex/azure-databricks-connect-azure-storage-to-databricks-285cac910fa0

# Bronze Layer



# Use Case – Applicative Flow Medallion Architecture

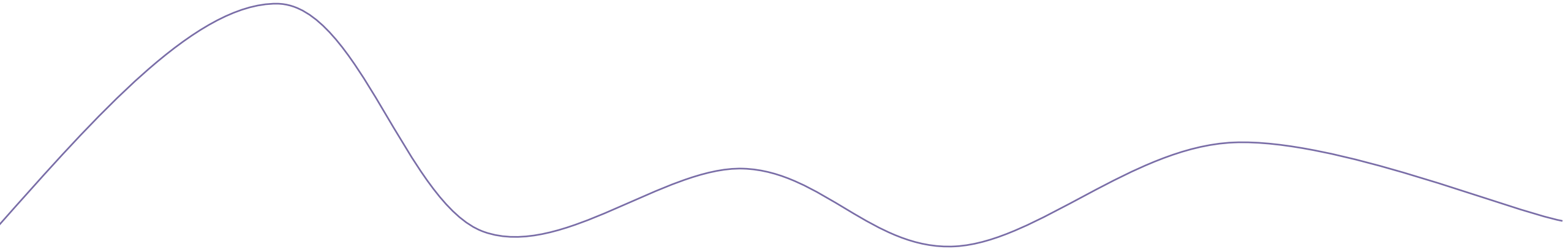


# UC 01 – Init Ingest

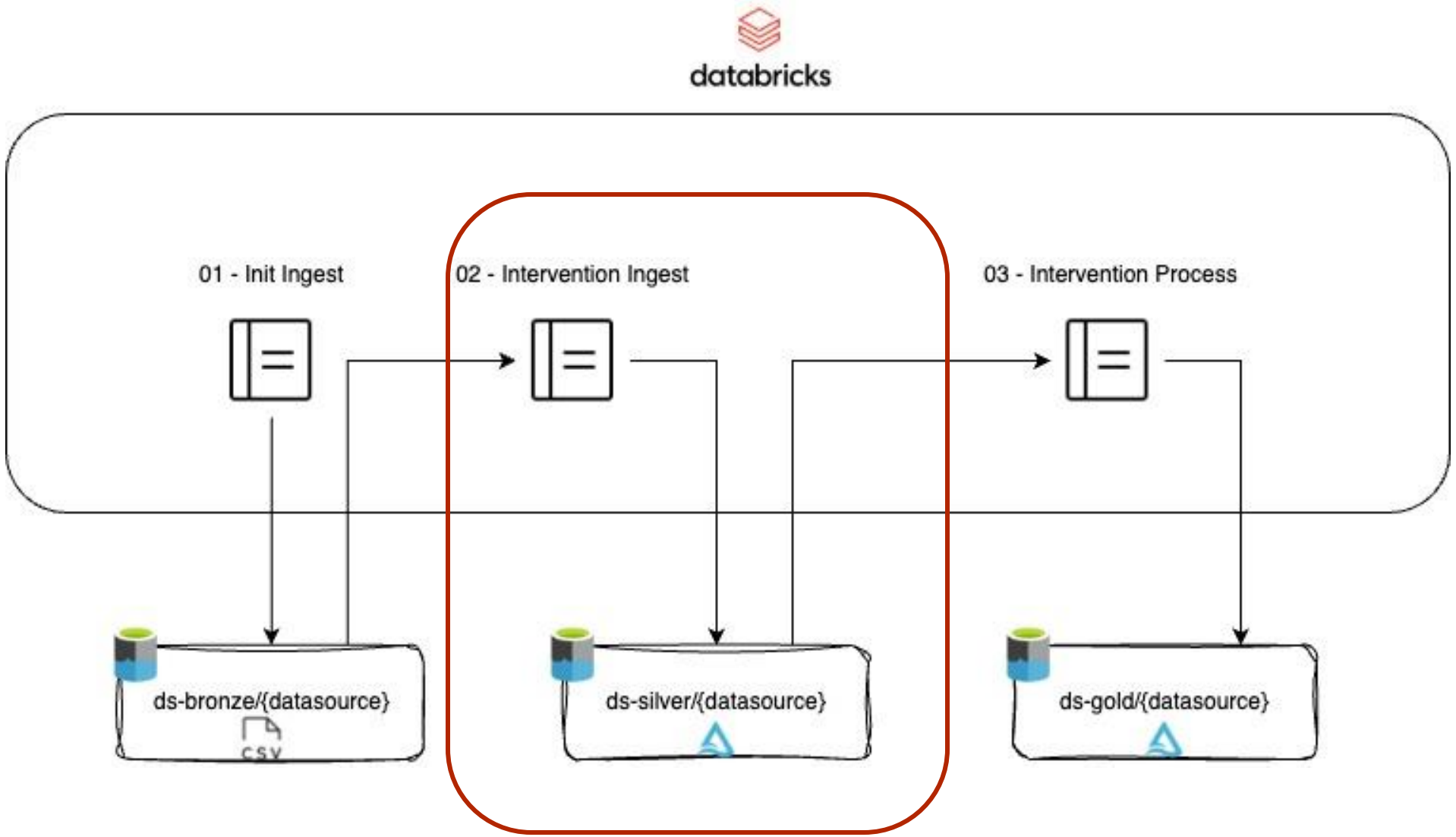



- Create 3 containers named: ds-bronze, ds-silver, and ds-gold.
- Set up a mount point for each of these containers.
- Upload the 3 data sources with the following structure:
  - ds-bronze
  - /insee/current/INSEE.csv
  - /siren/ current /siren.csv
  - /intervention/ current /intervention.csv
- Create a database named "bronze"
- Create the associated tables:
  - bronze.insee
  - bronze.siren
  - bronze.intervention

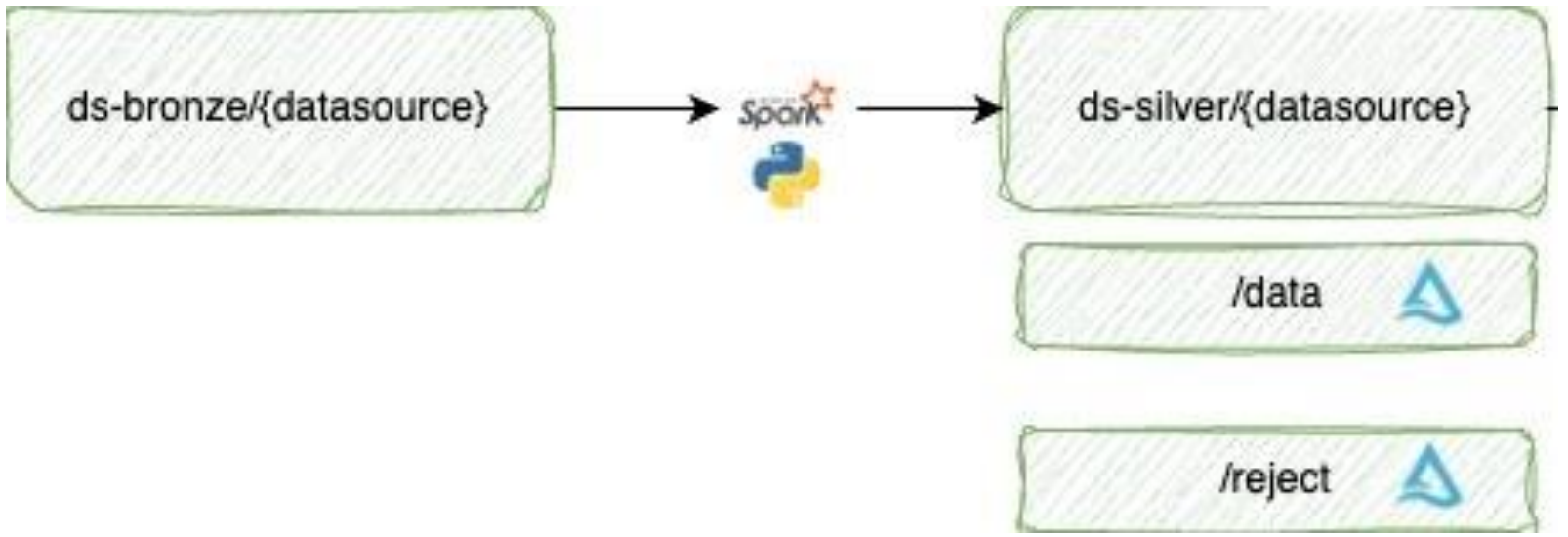
# Silver Layer



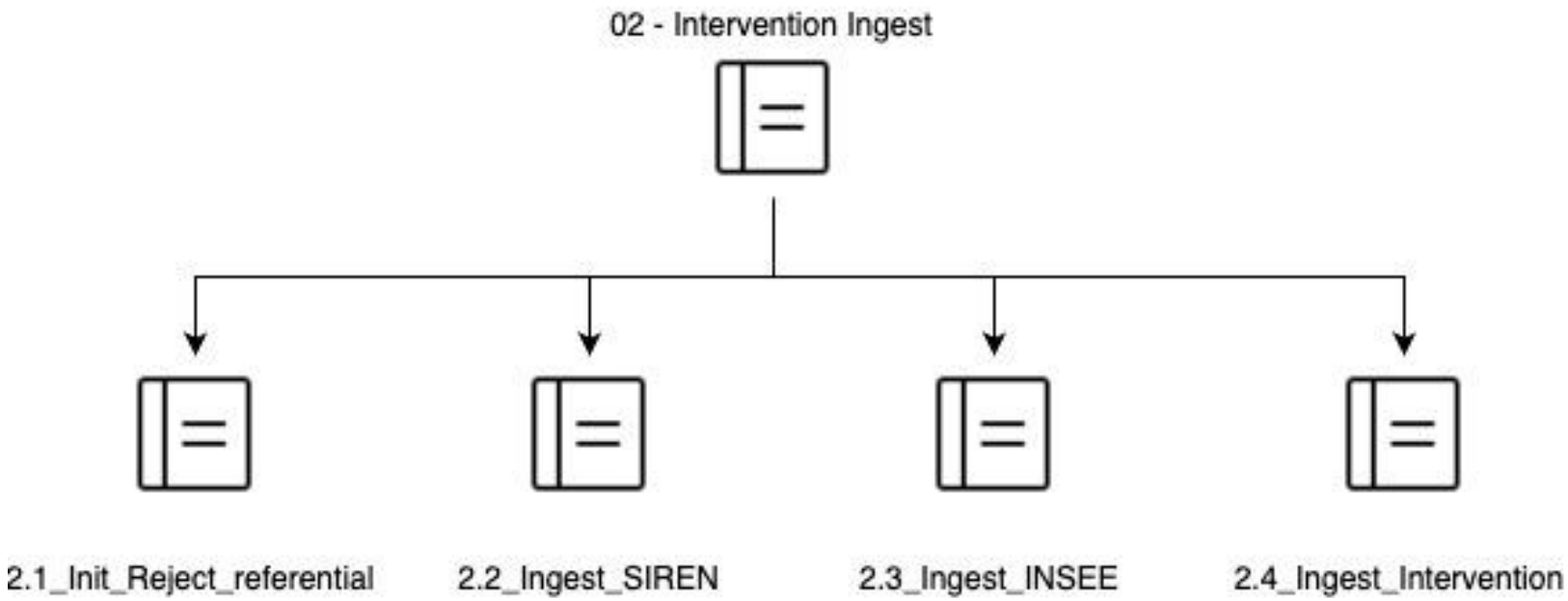
# Use Case – Applicative Flow Medallion Architecture



# Data Quality



# Intervention Ingest - Scheduling



# UC 02 – Intervention Ingest




- Use the template Notebook "02 - Intervention Ingest.dbc".

02 - Intervention Ingest

- Create a database named "silver"

2.1_Init_Reject_referential

- Create the managed table "silver.referential_reject". This table must include the following metadata:

| IdReject | targetTable | rejectCause |
|---|---|---|
| Int | VARCHAR(25) | VARCHAR(100) |

- Create the script to truncate the 'silver.referential_reject ' table, followed by the insertion of all existing causes (next slides).

# UC 02 – Intervention Ingest




| IdReject | targetTable | rejectCause | To be deleted |
|---|---|---|---|
| 1000 | Intervention | Intervention dupliqué | True |
| 2000 | SIREN | SIREN dupliqué | True |
| 3000 | INSEE | INSEE dupliqué | True |

# UC 02 – Intervention Ingest




| IdReject | targetTable | rejectCause | To be deleted |
|---|---|---|---|
| 1001 | Intervention | Intervention manquante | True |
| 1002 | Intervention | Dossier manquant | True |
| 2001 | Intervention | Code SIREN manquant | True |
| 2002 | SIREN | Code SIREN manquant | True |
| 3001 | Intervention | Code INSEE manquant | True |
| 3002 | INSEE | Code INSEE manquant | True |
| 4001 | Intervention | Code état du dossier manquant | True |
| 5001 | Intervention | Code CA | True |
| 6001 | Intervention | Code criticité manquant | True |

# SIREN Table - light

| SIREN |
| --- |
| SIREN<br>NIC<br>siege<br>enseigne |

# UC 02 – Intervention Ingest




- Inspired by "2.4_Ingest_Intervention”’s Notebook , do the same for INSEE & SIREN.

**2.2_Ingest_SIREN**

- CREATE OR REPLACE TEMP VIEW siren_duplicated
- CREATE OR REPLACE TABLE silver.siren_reject (Location: '/mnt/ds-silver/siren/reject')
- INSERT INTO silver. siren_reject (IdReject 2002 & 2000)
- CREATE OR REPLACE TABLE silver.siren (Location: '/mnt/ds-silver/siren/data')
- INSERT INTO silver.siren (without IdReject 2002 & 2000)

**2.3_Ingest_INSEE**

- CREATE OR REPLACE TEMP VIEW insee_duplicated
- CREATE OR REPLACE TABLE silver.insee_reject (Location: '/mnt/ds-silver/insee/data')
- INSERT INTO silver.insee_reject (IdReject 3002 & 3000)
- CREATE OR REPLACE TABLE silver.insee (Location: '/mnt/ds-silver/insee/data')
- INSERT INTO silver.insee (without IdReject 3002 & 3000)

# UC 02 – Intervention Ingest



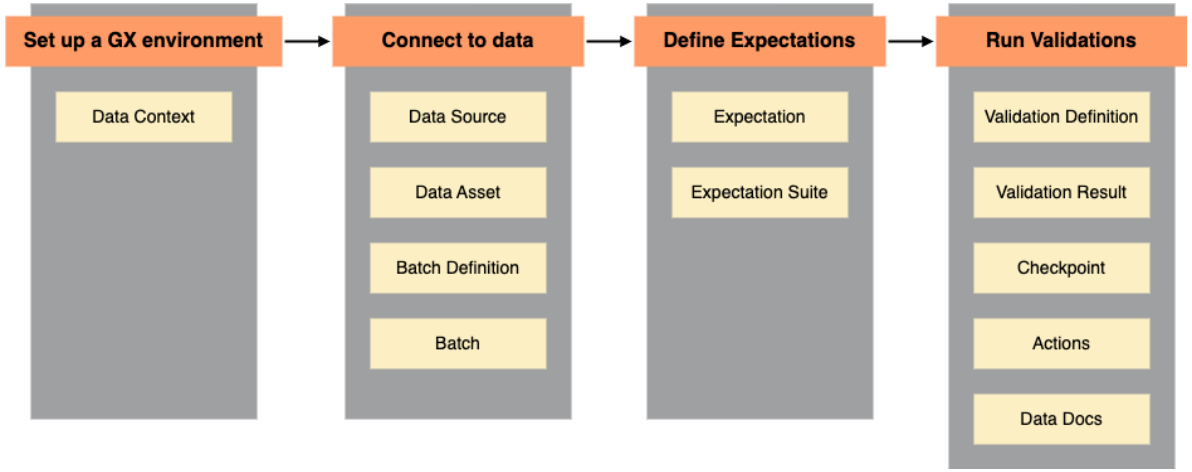

2.2_Ingest_SIREN

- Add TRUNCATE TABLE silver.siren after creation

2.3_Ingest_INSEE

- Add TRUNCATE TABLE silver.insee after creation

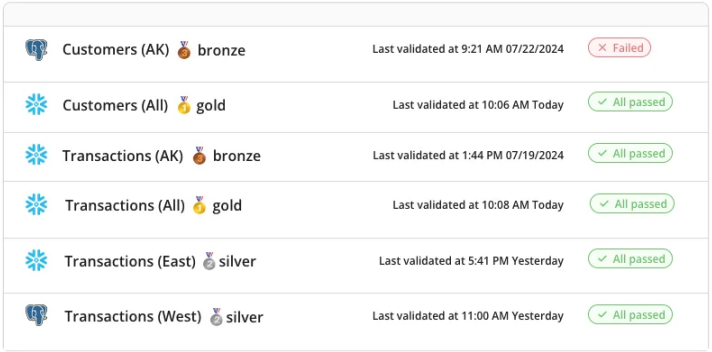
# Great Expectation




great expectations

```
pip install great_expectations

preset_expectation =
gx.expectations.ExpectColumnMaxToBeBetween(
   column="passenger_count", min_value=1,
max_value=6
)
```




https://docs.greatexpectations.io/docs/core/introduction/

# SODA

### Data observability.

Monitor data in production, raise alerts, and help debug data and pipelines to find the root cause as fast possible.

### Pipeline testing.

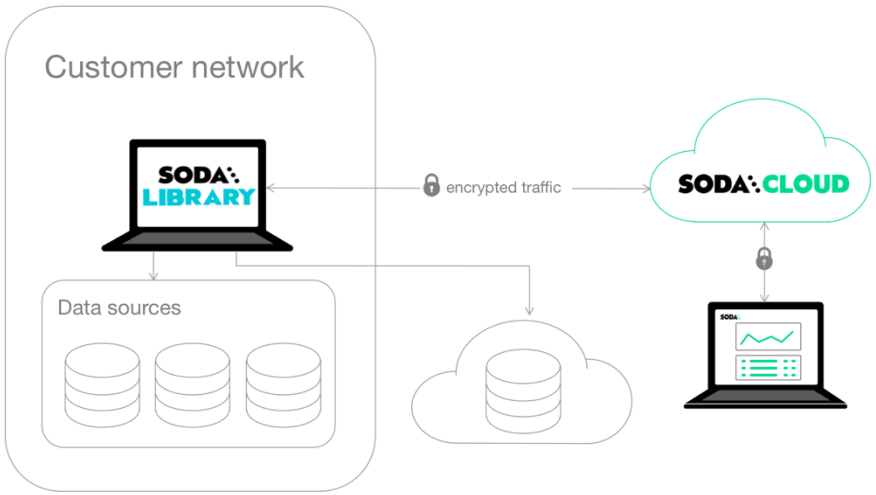
Test data as early as possible in your pipelines and CI/CD workflows, to avoid merging bad-quality data into production.

### No-code checks.

Empower business users to contribute to data quality and maintain standards with Soda AI assistants.

### Operational data quality.

Streamline data processes and route issue alerts to the appropriate data owners for swift resolution.



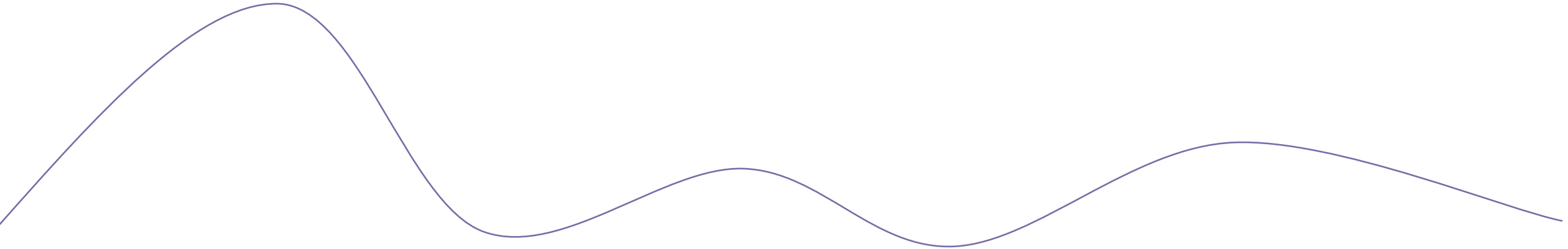

```
checks for employee_info:
 - invalid_count(Department) = 0:
     valid values: ['Sales', 'Research & Development', 'Human Resources']
     name: Only correct departments are present in the dataset
     attributes:
       dimension: [Validity]
       pipeline_stage: Ingest
       team: Data Engineering
 - missing_count(EmployeID) = 0:
     name: No null values in the Employee ID column
     attributes:
       dimension: [Completeness]
       pipeline_stage: Ingest
       team: Data Engineering
 - duplicate_count(EmployeeID) = 0:
     name: No duplicate IDs
     attributes:
       dimension: [Uniqueness]
       pipeline_stage: Ingest
       team: Data Engineering
```

SODA

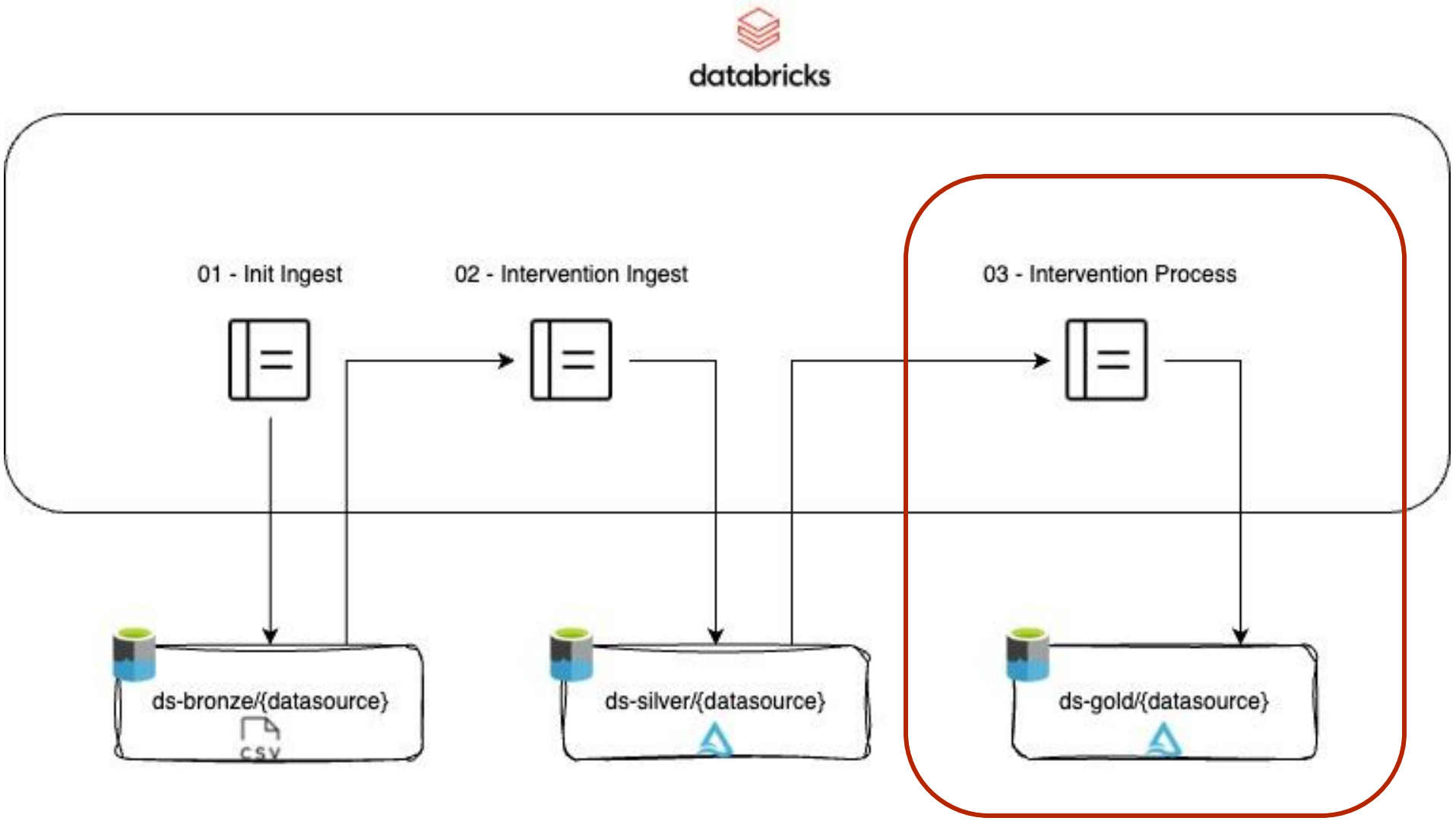


https://docs.soda.io/

# Gold Layer



# Use Case – Applicative Flow Medallion Architecture



# Dimensions tables

| gold.dim_siren |
| --- |
| IDSIREN (PK) |
| codeSIREN |
| NIC |
| siege |
| enseigne |

| gold.dim insee |
| --- |
| IDINSEE (PK) |
| codeINSEE |
| commune |
| departement |
| region |

| gold.dim criticality |
| --- |
| IDCriticality (PK) |
| code |
| libelle |

| gold.dim_status |
| --- |
| IDStatus (PK) |
| code |
| libelle |

| gold.dim_date |
| --- |
| IDDate (PK) |
| date |
| IdSemaine |
| semaine |
| IdMois |
| mois |
| IdMoisAnnee |
| annee mois |
| annee |
| dateLongFormat |

| gold.dim_agency |
| --- |
| IDAgency (PK) |
| code |
| libelle |

# Table de Transco – état du dossier

| Code état dossier | Etat dossier |
|---|---|
| 1 | Créé |
| 2 | Transmis |
| 3 | Modifié |
| 4 | Clôt |

# Table de Transco – criticité

| Code criticité | Libellé |
|---|---|
| 18 | Critique |
| 19 | Haut |
| 22 | Moyen |
| 60 | Bas |
| 61 | A planifier |

# UC 03 – Intervention Process




03 - Intervention Process

- Create a database named "gold"

3.1_Process_dimensions

- Create each dimension (cf. Dimensions tables)
- Create an automated script to populate the Date dimension. The first date to be inserted will be 01/01/2010, with IdDate 20100101, and the last date will be 31/12/2030.
- Create the initialization procedure for status (état du dossier) and criticality (criticité).
- Create the insertion procedure (without allowing modifications – only new rows) for agency (CA : centre agence).
- For the SIREN & INSEE dimension, use an SCD1.

# UC 03 – Intervention Process




03 - Intervention Process

- Create a database named "gold"

3.1_Process_fact

- Create the fact table, using each dimension surrogate key

# Change data feed

The Change Data Feed (CDF) feature allows Delta tables to track row-level changes between versions of a Delta table. When enabled on a Delta table, the runtime records "change events" for all data written to the table.

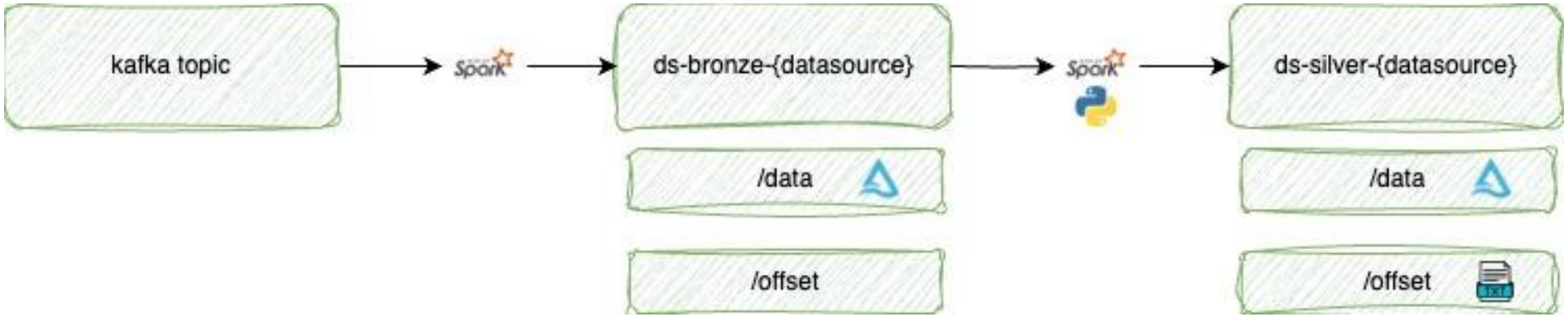
SQL

```
set spark.databricks.delta.properties.defaults.enableChangeDataFeed = true;
```

```
# providing a starting version
spark.readStream.format("delta") \
  .option("readChangeFeed", "true") \
  .option("startingVersion", 0) \
  .table("myDeltaTable")
```

| Column name | Type | Values |
|---|---|---|
| _change_type | String | insert, update_preimage , update_postimage, delete (1) |
| _commit_version | Long | The Delta log or table version containing the change. |
| _commit_timestamp | Timestamp | The timestamp associated when the commit was created. |

# Change data feed




https://docs.delta.io/latest/delta-change-data-feed.html